1 Descripción general del proyecto

Siqueiros es un software de restauración de fotografías antiguas desarrollado en **MATLAB App Designer** (clase `app1 < matlab.apps.AppBase`, archivo `app1.mlapp`). Recibe una fotografía dañada, aplica algoritmos deterministas de procesamiento digital de imágenes para detectar y corregir el daño, y presenta lado a lado la imagen original y la imagen restaurada, cada una con su histograma [1].

Objetivo. Integrar en una sola aplicación las técnicas de procesamiento digital de imágenes necesarias para reducir rayones, atenuar arrugas y pliegues, disminuir manchas, reconstruir regiones faltantes, corregir la degradación causada por el tiempo y adicionar color a imágenes en escala de grises. El proyecto no emplea modelos de inteligencia artificial: toda decisión de reparación depende de máscaras, umbrales y filtros definidos en el código.

Funcionamiento general. El usuario importa una imagen, elige un módulo de restauración y revisa el resultado en el visor derecho junto con su histograma. La salida de un módulo se convierte en la entrada del siguiente, por lo que los módulos pueden aplicarse sucesivamente y en el orden que convenga a cada fotografía. Las operaciones se pueden deshacer y rehacer, y el resultado se exporta a disco en el formato elegido.

Módulos de restauración

Módulo	Daño que atiende	Técnica principal
M1 · Rayones	Rayaduras claras y finas	Top-hat morfológico, umbral adaptativo y relleno por interpolación (`regionfill`)
M2 · Arrugas y pliegues	Arrugas, pliegues y variaciones locales de brillo	Cierre morfológico y filtro de mediana 5 × 5
M3 · Manchas	Manchas y gotas oscuras o con diferencia de color	Detección en CIE L*a*b*, histéresis, filtrado por forma e inpainting coherente
M4 · Reconstrucción	Zonas rotas o faltantes (esquinas y roturas)	Máscara por brillo en el perímetro y top-hat, relleno por interpolación
M5 · Degradación	Bajo contraste, opacidad y amarillamiento	Saturación en HSV, ecualización adaptativa CLAHE y suavizado gaussiano
M6 · Adición de color	Imágenes en escala de grises	Curvas de transferencia cromática en L*a*b* (interpolación `pchip`)

Todos los módulos operan tanto en imágenes en **escala de grises** como a **color (RGB)**. La aplicación importa archivos **PNG, JPG/JPEG, TIFF y BMP** y exporta en **PNG, JPG, TIF y BMP**.

2 Descripción de los extras añadidos

Además de lo solicitado en los lineamientos del proyecto, la aplicación incorpora:

- **Rehacer** (`RehacerButton`): complementa al botón Deshacer y permite avanzar de nuevo en el historial.
- **Historial de operaciones multinivel:** cada operación guarda una copia completa de la imagen; Deshacer y Rehacer recorren estos estados sin un límite explícito de pasos.
- **Ventana “Historial de Operaciones”:** muestra miniaturas paginadas (tres por página) con el nombre de cada paso e indica cuál es el estado actual.

- **Etiqueta “Estado de la imagen”:** informa en todo momento qué acción se ejecuta o cuál fue su resultado (por ejemplo, el número de manchas reparadas y el porcentaje de imagen modificado).
- **Histograma doble:** se muestra el histograma de la imagen original y el de la imagen restaurada al mismo tiempo, para comparar el efecto de cada módulo.
- **Reparación de manchas con respaldo:** M3 usa `inpaintCoherent` y, si esa función no está disponible, recurre automáticamente a `regionfill`.
- **Identidad visual propia:** logotipo “Siqueiros”, lema “Restaura hoy, conserva siempre” y una paleta de colores (azul marino, crema, café y beige) aplicada a toda la interfaz.

3 Documentación técnica

3.1 Entorno y dependencias

Elemento	Especificación
Lenguaje y plataforma	MATLAB con App Designer. Archivo de diseño `app1.mlapp`; el código fuente se incluye completo en la sección 7. El formato `.mlapp` es un paquete ZIP/OPC propio de App Designer [2].
Versión	Paquete versión 1.0. La metadata declara como versión mínima R2018a; se recomienda **R2019a o posterior**, pues `inpaintCoherent` se introdujo en esa versión [7] y la ventana de historial usa `uigridlayout`.
Toolbox	Image Processing Toolbox (`strel`, `imtophat`, `regionfill`, `adapthisteq`, `rgb2lab`, `lab2rgb`, `inpaintCoherent`, `imhist`, entre otras).
Recursos gráficos	Carpeta `Recursos/` junto a `app1.mlapp`, con `logofondo.png`, `2.png`, `3.png` y `4.png` (logotipo e iconos de la navegación lateral).
Fuentes tipográficas	Century Gothic, Bahnschrift y Bauhaus 93. Si no están instaladas, MATLAB sustituye la fuente y el aspecto puede variar.
Ventana	Tamaño 1090 × 851 unidades; panel central de 897 × 790.

3.2 Arquitectura y flujo de datos

La aplicación es una clase de App Designer: las propiedades públicas contienen los componentes visuales; las privadas guardan el estado de trabajo; y los métodos privados atienden los callbacks de los botones (`ButtonPushedFcn`) y el refresco de la pantalla [1]. Los pasos de procesamiento se ejecutan de forma secuencial al pulsar cada botón.

Estado interno (propiedades privadas)

Propiedad	Función
`ImagenOriginal`	Imagen cargada desde disco. Es la referencia: alimenta el visor y el histograma de la izquierda y nunca se modifica.
`ImagenProcesada`	Versión sobre la que operan los módulos y la que se exporta.
`HistorialImagenes`	Celda con una copia completa de la imagen en cada estado.
`HistorialNombres`	Celda con el rótulo de la operación asociada a cada estado.
`IndiceHistorial`	Posición activa del historial; determina el estado visible y los límites de Deshacer y Rehacer.

Funciones auxiliares

- `guardarEstado(app, nuevaImagen, nombreOperacion)`: si se está en medio del historial, descarta los estados de “rehacer”; añade la nueva imagen, incrementa el índice y la asigna a `ImagenProcesada`. Si no recibe nombre, usa el rótulo genérico “Operación n”.
- `actualizarInterfazProcesada(app)`: dibuja la imagen procesada en `UIAxes_2` y los histogramas en `UIAxes_4` (restaurada) y `UIAxes_3` (original). Si la imagen es RGB, el histograma se calcula tras convertirla a gris con `rgb2gray` [8]; el eje X se fija en 0–255.



Figura 1. Flujo principal de datos. Cada módulo trabaja sobre el estado procesado vigente, guarda una copia en el historial y actualiza el visor restaurado y los histogramas.

3.3 Componentes de la interfaz

Componente	Descripción
`UIAxes`	Visor de la imagen original (izquierda, 387 × 321).
`UIAxes_2`	Visor de la imagen restaurada (derecha, 387 × 321).
`UIAxes_3`	Histograma de la imagen original (387 × 158).
`UIAxes_4`	Histograma de la imagen restaurada (387 × 158).
Panel “Herramientas de Restauración”	Seis botones de módulos: Rayones, Arrugas / Pliegues, Manchas, Reconstrucción, Degradación y Adición de Color.
Fila de acciones	Importar, Deshacer, Rehacer y Exportar.
Navegación lateral	Logotipo y botones Inicio, Historial y Ayuda.
`EstadodelaimagenLabel`	Etiqueta de estado inferior izquierda.

Paleta de colores. Se tomó de las propiedades de los componentes del código:

Azul marino	Crema	Café oscuro	Beige de borde
#0F4066	#F2E8D9	#47381F	#D9C7A8
RGB 15, 64, 102	RGB 242, 232, 217	RGB 71, 56, 31	RGB 217, 199, 168

Los botones de acción y de restauración usan azul marino con texto crema; el panel y los botones laterales, crema con texto café.

3.4 Algoritmos de los módulos de restauración

Todos los parámetros están fijados en el código; el usuario selecciona un módulo, pero la interfaz no incluye controles para modificar umbrales ni radios. Cada módulo toma la imagen de `ImagenProcesada` y, al terminar sin error, guarda el resultado en el historial y refresca los visores.

M1 Eliminación de rayones

Callback: RayonesButtonPushed

Detecta estructuras claras y delgadas (rayones) y las rellena a partir de su entorno.

1. Convierte la imagen a gris con `rgb2gray` si es RGB.
2. Aplica top-hat morfológico con `strel('disk', 6)`. El top-hat resalta estructuras claras pequeñas respecto al elemento estructurante [3].
3. Calcula el umbral `mediana + 1.8 × desv. estándar` y binariza: `tophat > umbral`.
4. Dilata la máscara con `strel('disk', 2)` para cubrir los bordes desgastados del rayón.
5. Rellena las zonas enmascaradas con `regionfill`, canal por canal si la imagen es RGB; la interpolación parte del contorno de la máscara [4].

M2 Eliminación de arrugas y pliegues

Callback: ArrugasPlieguesButtonPushed

Atenúa variaciones locales de brillo producidas por pliegues mediante un suavizado morfológico y de mediana.

1. Define el elemento estructurante `strel('disk', 3)`.
2. Aplica cierre morfológico (`imclose`) a la imagen en gris o a cada canal RGB por separado.
3. Aplica un filtro de mediana de 5 × 5 (`medfilt2`, con extensión simétrica de bordes) al resultado del cierre.
4. Guarda el estado con el rótulo “M2 - Arrugas y pliegues”.

> **Observación**
> El procesamiento es un suavizado global: no selecciona localmente los pliegues, por lo que puede reducir detalle fino junto con el defecto.

M3 Disminución de manchas

Callback: ManchasButtonPushed

Detecta manchas y gotas comparando cada píxel con un fondo local estimado, y las repara con inpainting coherente.

1. Normaliza la imagen a `uint8`; si es gris, la replica en tres canales para trabajar en color.
2. Convierte a CIE L*a*b* con `rgb2lab` [6] y separa L (luminancia), a y b (croma).
3. Estima el fondo local de cada canal reduciendo la imagen al 25 %, filtrando con mediana de 25 × 25 y reescalando al tamaño original.
4. Calcula la desviación de luminancia (`dL`) y la diferencia cromática (`dC`) respecto del fondo.

5. Normaliza con escalas robustas (basadas en la mediana) y obtiene un puntaje por píxel; en luminancia solo cuentan los oscurecimientos.

6. Extiende por histéresis: las semillas (`puntaje > kFuerte`) se expanden a toda la región con puntaje > `kDebil` mediante `imreconstruct`.

7. Descarta regiones por tamaño (`areaMin`, `areaMaxFr`) y por forma (`ratioMax`, `solidezMin`): protege líneas y estructuras alargadas.

8. Dilata la máscara con radio `radioDilat` para cubrir el halo difuso y repara con `inpaintCoherent` (`'Radius', 15, 'SmoothingFactor', 2`) [7]; si la llamada falla, usa `regionfill` canal por canal.

9. Si la imagen era gris, devuelve un solo canal. El estado muestra cuántas manchas se repararon y el porcentaje de imagen modificado.

Parámetro	Valor	Función
kFuerte	3.0	Sigmas para las semillas (menor valor = mayor sensibilidad)
kDebil	1.8	Sigmas para extender la mancha (histéresis)
factorEsc	0.25	Escala para estimar el fondo
ventanaEsc	25	Ventana de mediana (impar); ≈ 100 px reales a esa escala
areaMin	30	Píxeles mínimos para considerar una mancha
areaMaxFr	0.010	Fracción máxima de la imagen por región
ratioMax	3	Eje mayor / menor; superior a este valor se protege (línea o rayo)
solidezMin	0.5	Las formas muy irregulares se protegen
radioDilat	5	Radio de dilatación de la máscara
radioInp	15	Radio de `inpaintCoherent`

> **Observación**
> M3 está orientado a manchas o gotas oscuras o con diferencia cromática respecto del fondo local. Fotografías con texturas o daños distintos pueden requerir calibrar estos parámetros en el código.

M4 Reconstrucción de pérdida de contenido

Callback: ReconstruccinButtonPushed

Reconstruye esquinas y bordes faltantes y roturas finas, limitando la intervención al perímetro de la imagen.

1. Convierte a gris y detecta papel o fondo faltante como píxeles claros: `gris > 225`.

2. Construye una máscara de perímetro que abarca el 25 % exterior de filas y columnas; así protege el centro de la imagen (rostros y detalles principales). La máscara de esquinas es la intersección de ambas.

3. Detecta grietas con top-hat de `strel('disk', 4)` y umbral `mediana + 3 × desv. estándar` [3].

4. Une las máscaras, elimina componentes menores de 30 píxeles (`bwareaopen`) y dilata con `strel('disk', 3)`.

5. Si no se detecta ninguna rotura, informa en el estado y termina sin modificar la imagen.

6. Rellena con `regionfill` canal por canal [4].

Observación
La detección se basa en umbrales de brillo y en la periferia de la imagen, no en una máscara dibujada por el usuario. El umbral 225 supone intensidades de 8 bits; en imágenes de 16 bits debe recalibrarse.

M5 Restauración por degradación

Callback: DegradacinButtonPushed

Recupera contraste y viveza en fotografías opacas, amarillentas o de bajo contraste.

1. Parte de `ImagenProcesada`; si no existiera, usa `ImagenOriginal`.
2. En color: convierte a HSV (`rgb2hsv`), multiplica la saturación por 1.3 con límite 1 y ecualiza adaptativamente el canal V con `adapthisteq` y `'ClipLimit', 0.02` [5].
3. En escala de grises: aplica `adapthisteq` directamente a la imagen.
4. Aplica un filtro gaussiano suave (`imgaussfilt, σ = 0.6`) y convierte la salida a `uint8`.
5. CLAHE calcula los ajustes por teselas y limita la amplificación del contraste [5].

M6 Adición de color

Callback: AdicindeColorButtonPushed

Colorización tonal determinista: asigna croma a cada píxel según su luminancia.

1. Obtiene la luminancia en gris y la escala a L^* de 0 a 100 (`L = lum × 100`).
2. Define siete puntos de control para las curvas de croma (a^*: verde–rojo; b^*: azul–amarillo) [6].
3. Interpola cada curva con `interp1(..., 'pchip')` para todos los píxeles según su brillo.
4. Suaviza las matrices a^* y b^* con `imgaussfilt, σ = 1.5` para homogeneizar la transición cromática.
5. Reconstruye RGB con `lab2rgb` y convierte a `uint8`.

Puntos de control	Valores
L_puntos (L^*, brillo)	0, 12, 30, 55, 75, 92, 100
a_puntos (a^*, verde–rojo)	1, 2, 7, 13, 9, 2, 0
b_puntos (b^*, azul–amarillo)	2, 3, 10, 16, 11, 3, 1

Observación
Es una colorización tonal fija: no predice los colores reales de cada objeto. Si la imagen ya es a color, se convierte primero a gris.

3.5 Callbacks e historial

Callback	Responsabilidad
ImportarButtonPushed	Abre `uigetfile` con filtros PNG, JPG/JPEG, TIFF y BMP, lee con `imread`, inicializa imágenes e historial (índice 1, "Imagen original") y dibuja el visor y los histogramas.
RayonesButtonPushed · ArrugasPlieguesButtonPushed · ManchasButtonPushed	Ejecutan los módulos M1 a M3, guardan el nuevo estado y refrescan la interfaz.
ReconstruccinButtonPushed · DegradacinButtonPushed · AdicindeColorButtonPushed	Ejecutan los módulos M4 a M6. Usan `try / catch` para mostrar errores en la etiqueta de estado o en un cuadro de diálogo.
DeshacerButtonPushed · RehacerButtonPushed	Retroceden o avanzan una posición el índice del historial y restauran la imagen almacenada; si no hay más pasos, muestran un aviso.
HistorialButtonPushed	Crea una ventana de 650 × 540 con hasta tres miniaturas por página, nombre de cada paso y navegación Anterior / Siguiente.
ExportarButtonPushed	Abre `uiputfile` (PNG, JPG, TIF o BMP; nombre sugerido `imagen_restaurada.png`) y escribe la imagen procesada con `imwrite`.

Comportamiento del historial. Al importar, el historial contiene solo la imagen original y el índice vale 1. Deshacer y Rehacer únicamente mueven el índice; al aplicar una operación después de deshacer, `guardarEstado` elimina la rama futura. El historial existe solo en memoria durante la sesión.

3.6 Limitaciones técnicas

- Las máscaras y los parámetros son heurísticos y no se pueden ajustar desde la interfaz; los defectos no detectados o una máscara demasiado extensa pueden dejar artefactos.
- Cada entrada del historial es una copia completa de la imagen y no hay límite de pasos, por lo que el consumo de memoria crece con el tamaño de las fotos y el número de operaciones.
- Los histogramas fijan el eje X en 0–255, por lo que imágenes de 16 bits pueden mostrarse recortadas.
- Los botones **Inicio** y **Ayuda** no tienen callback asociado en la versión actual; **Historial** solo permite consultar los pasos, no restaurar uno directamente.
- La exportación escribe únicamente la imagen procesada actual.

4 Documentación del usuario

4.1 Especificaciones técnicas para el usuario

Especificación	Detalle
Requisitos	MATLAB R2019a o posterior con Image Processing Toolbox. Carpeta `Recursos` en la misma ubicación que `app1.mlapp`.
Formatos de entrada	PNG, JPG / JPEG, TIFF (.tif, .tiff) y BMP.
Formatos de salida	PNG, JPG, TIF y BMP.
Tipos de imagen	Escala de grises y color (RGB) en todos los módulos.
Control	Todo se maneja con el mouse mediante botones.
Historial	En memoria, durante la sesión. Los pasos se pueden deshacer, rehacer y consultar.

4.2 Cómo usar la aplicación

1 — Abrir la aplicación
Abra `app1.mlapp` en MATLAB con “Run” (o ejecute `app1` en la ventana de comandos).

2 — Importar una imagen
Pulse **Importar**, elija el archivo (PNG, JPG, TIFF o BMP) y confirme. La imagen aparece en el visor **Original** con su histograma, y también en **Imagen Restaurada**. La etiqueta de estado confirma la importación.

3 — Elegir una herramienta
En **Herramientas de Restauración** pulse el módulo que corresponde al daño. La etiqueta de estado indica que el proceso está en curso; espere a que muestre el mensaje de término.

4 — Revisar el resultado
Compare el visor derecho y su histograma con la referencia original de la izquierda.

5 — Encadenar operaciones
Pulse otro módulo: se aplicará sobre el resultado actual. Por ejemplo, quite primero las arrugas (M2) y después mejore el contraste (M5), o al revés.

6 — Deshacer o rehacer
Use **Deshacer** para volver al estado anterior y **Rehacer** para avanzar de nuevo.

7 — Consultar el historial
Pulse **Historial** para ver, en una ventana, las miniaturas de cada paso; el estado actual se marca con “Estado Actual”. Use **Anterior** y **Siguiente** para cambiar de página.

8 — Exportar
Pulse **Exportar**, elija carpeta, nombre y formato (PNG, JPG, TIF o BMP). Se guarda la imagen restaurada actual.

4.3 Descripción de los controles

Control	Acción
Importar	Carga una imagen desde el disco y reinicia el historial.
Rayones (M1)	Elimina rayones claros y finos.
Arrugas / Pliegues (M2)	Atenúa arrugas y pliegues. Puede suavizar detalle fino.
Manchas (M3)	Repara manchas y gotas. Informa cuántas encontró y el porcentaje modificado.
Reconstrucción (M4)	Rellena esquinas y roturas, en el perímetro de la imagen.
Degradación (M5)	Aumenta contraste y saturación en fotos opacas o amarillentas.
Adición de Color (M6)	Coloriza la imagen con una paleta tonal fija.
Deshacer / Rehacer	Retrocede o avanza un paso en el historial.
Exportar	Guarda la imagen restaurada actual en el formato elegido.
Historial	Abre la ventana de miniaturas de las operaciones realizadas.
Inicio · Ayuda	Sin función asociada en la versión actual; las instrucciones de uso están en este documento.
Estado de la imagen	Etiqueta inferior izquierda con el mensaje de la última acción.

4.4 Mensajes de la aplicación

Mensaje	Significado
“Falta Imagen”	Se pulsó un módulo sin haber importado una imagen. Importe una primero.
“No hay más operaciones para deshacer / rehacer”	Se llegó al inicio o al final del historial.
“Historial Vacío”	Se abrió el historial sin imagen importada.
“No se detectaron roturas / manchas”	El módulo no encontró daño con sus umbrales y la imagen no cambió. Para M3, se puede bajar `kFuerte` en el código.
“Sin imagen”	Se intentó exportar sin imagen cargada.
“Error en …”	Falló el módulo indicado; el cuadro muestra el mensaje de MATLAB.

4.5 Recomendaciones de uso

- Para fotografías con varios daños, encadene los módulos en el orden que dé mejor resultado y use Deshacer para comparar.
- M6 convierte la imagen a gris antes de colorizar; aplíquelo al final si desea conservar los colores originales en los demás pasos.
- En fotografías muy grandes, el historial consume más memoria: exporte los resultados intermedios importantes.

5 Capturas de pantalla de la aplicación

Las capturas muestran la interfaz de la aplicación en su estado inicial, antes de importar una fotografía.



Figura 2. Pantalla principal de Siqueiros: navegación lateral, visores Original e Imagen Restaurada, histogramas, herramientas de restauración y acciones.

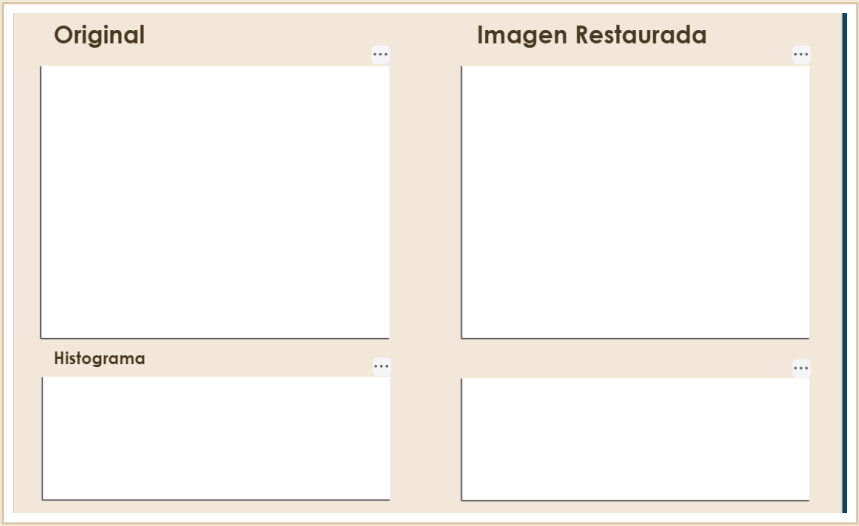


Figura 3. Zona central: visores superiores de imagen original y restaurada, e histogramas de intensidad en la fila inferior.



Navegación lateral
Agrupa el logotipo de Siqueiros y los botones Inicio, Historial y Ayuda. Historial abre la ventana con las miniaturas de las operaciones guardadas.

Figura 4. Marca y navegación lateral.



Figura 5. Barra inferior: seis módulos de restauración y controles para importar, deshacer, rehacer y exportar.

6 Bibliografía consultada

Documentación oficial de MathWorks consultada para la estructura de App Designer y las funciones de procesamiento utilizadas. Fechas de consulta: 10 de noviembre de 2026.

[1] MathWorks. *Develop Apps Using App Designer*. https://www.mathworks.com/help/matlab/app-designer.html

[2] MathWorks. *App Designer File Formats*. https://www.mathworks.com/help/matlab/creating_guis/app-designer-file-formats.html

[3] MathWorks. *imtophat — Top-hat filtering*. https://www.mathworks.com/help/images/ref/imtophat.html

[4] MathWorks. *regionfill — Fill specified regions in image using inward interpolation*. https://www.mathworks.com/help/images/ref/regionfill.html

[5] MathWorks. *adapthisteq — Contrast-limited adaptive histogram equalization (CLAHE)*. https://www.mathworks.com/help/images/ref/adapthisteq.html

[6] MathWorks. *rgb2lab — Convert RGB to CIE 1976 L*a*b**. https://www.mathworks.com/help/images/ref/rgb2lab.html

[7] MathWorks. *inpaintCoherent — Restore specific image regions using coherence transport based image inpainting*. https://www.mathworks.com/help/images/ref/inpaintcoherent.html

[8] MathWorks. *imhist — Histogram of image data*. https://www.mathworks.com/help/images/ref/imhist.html

7 Código fuente

Clase `app1` completa, tal como está en `app1.mlapp` (incluye los comentarios del código). Los comentarios aparecen en verde.

```
classdef app1 < matlab.apps.AppBase

    % Properties that correspond to app components
    properties (Access = public)
        UIFigure                        matlab.ui.Figure
        EstadodelaimagenLabel           matlab.ui.control.Label
        AyudaButton                     matlab.ui.control.Button
        Image_4                         matlab.ui.control.Image
        HistorialButton                 matlab.ui.control.Button
        Image_3                         matlab.ui.control.Image
        InicioButton                    matlab.ui.control.Button
        Image_2                         matlab.ui.control.Image
        Panel                           matlab.ui.container.Panel
        ExportarButton                  matlab.ui.control.Button
        RehacerButton                   matlab.ui.control.Button
        DeshacerButton                  matlab.ui.control.Button
        ImportarButton                  matlab.ui.control.Button
        Panel_2                         matlab.ui.container.Panel
        AdicindeColorButton             matlab.ui.control.Button
        DegradacinButton                matlab.ui.control.Button
        ReconstruccinButton             matlab.ui.control.Button
        ManchasButton                   matlab.ui.control.Button
        ArrugasPlieguesButton           matlab.ui.control.Button
        RayonesButton                   matlab.ui.control.Button
        HerramientasdeRestauracinLabel  matlab.ui.control.Label
        Label_3                         matlab.ui.control.Label
        ImagenRestauradaLabel           matlab.ui.control.Label
        OriginalLabel                   matlab.ui.control.Label
        Label                           matlab.ui.control.Label
        Label_2                         matlab.ui.control.Label
        UIAxes_4                        matlab.ui.control.UIAxes
        UIAxes_3                        matlab.ui.control.UIAxes
        UIAxes                          matlab.ui.control.UIAxes
        UIAxes_2                        matlab.ui.control.UIAxes
        Image                           matlab.ui.control.Image
    end


    properties (Access = private)
        ImagenOriginal  % Almacena la imagen importada
        ImagenProcesada % Almacena el resultado tras aplicar filtros/módulos

        % --- Pilas para Deshacer y Rehacer ---
        HistorialImagenes = {} % Celda para almacenar los estados de la imagen
        HistorialNombres = {}  % <--- AGREGAR ESTA LÍNEA
        IndiceHistorial = 0    % Posición actual en el historial
    end

    methods (Access = private)
           % Guarda el estado actual en el historial
        function guardarEstado(app, nuevaImagen, nombreOperacion)
            % Si no se especifica el nombre de la operación, usar uno por defecto
            if nargin < 3 || isempty(nombreOperacion)
                nombreOperacion = sprintf('Operación %d', app.IndiceHistorial + 1);
            end

            % Elimina estados de rehacer si nos encontramos en medio del historial
            if app.IndiceHistorial < length(app.HistorialImagenes)
                app.HistorialImagenes = app.HistorialImagenes(1:app.IndiceHistorial);
                app.HistorialNombres = app.HistorialNombres(1:app.IndiceHistorial);
            end

            % Agregar el nuevo estado y actualizar el índice
            app.IndiceHistorial = app.IndiceHistorial + 1;
            app.HistorialImagenes{app.IndiceHistorial} = nuevaImagen;
```

```
66              app.HistorialNombres{app.IndiceHistorial} = nombreOperacion;
67              app.ImagenProcesada = nuevaImagen;
68          end
69
70          % Actualiza los visores UIAxes_2 y UIAxes_4
71          function actualizarInterfazProcesada(app)
72              img = app.ImagenProcesada;
73
74              % 1. Renderizar Imagen Procesada
75              imshow(img, 'Parent', app.UIAxes_2);
76
77              % 2. Renderizar Histograma Restaurado en UIAxes_4
78              if size(img, 3) == 3
79                  grisHist = rgb2gray(img);
80              else
81                  grisHist = img;
82              end
83
84              [counts, binLocations] = imhist(grisHist);
85
86              cla(app.UIAxes_4);
87              bar(app.UIAxes_4, binLocations, counts, 'BarWidth', 1, 'FaceColor', [0 0.4470 0.7410], 'EdgeColor',
                    'none');
88              app.UIAxes_4.XLim = [0 255];
89              app.UIAxes_4.YLimMode = 'auto';
90              app.UIAxes_4.YColor = [0.9 0.9 0.9];
91              app.UIAxes_4.XColor = [0.9 0.9 0.9];
92
93              % 3. Garantizar Histograma Original en UIAxes_3
94              if ~isempty(app.ImagenOriginal)
95                  if size(app.ImagenOriginal, 3) == 3
96                      grisOrig = rgb2gray(app.ImagenOriginal);
97                  else
98                      grisOrig = app.ImagenOriginal;
99                  end
100                 [countsOrig, binLocOrig] = imhist(grisOrig);
101                 cla(app.UIAxes_3);
102                 bar(app.UIAxes_3, binLocOrig, countsOrig, 'BarWidth', 1, 'FaceColor', [0 0.4470 0.7410],
                        'EdgeColor', 'none');
103                 app.UIAxes_3.XLim = [0 255];
104                 app.UIAxes_3.YLimMode = 'auto';
105                 app.UIAxes_3.YColor = [0.9 0.9 0.9];
106                 app.UIAxes_3.XColor = [0.9 0.9 0.9];
107             end
108         end
109     end
110
111
112     % Callbacks that handle component events
113     methods (Access = private)
114
115         % Button pushed function: ImportarButton
116         function ImportarButtonPushed(app, event)
117             fileFilters = {
118                 '*.png;*.jpg;*.jpeg;*.tif;*.tiff;*.bmp', 'Archivos de imagen (*.png, *.jpg, *.jpeg, *.tif,
                        *.bmp)'; ...
119                 '*.png',  'Imágenes PNG (*.png)'; ...
120                 '*.jpg;*.jpeg', 'Imágenes JPEG (*.jpg, *.jpeg)'; ...
121                 '*.tif;*.tiff', 'Imágenes TIFF (*.tif, *.tiff)'; ...
122                 '*.bmp',  'Imágenes Bitmap (*.bmp)'; ...
123                 '*.*',    'Todos los archivos (*.*)'};
124             [fileName, folderPath] = uigetfile(fileFilters, 'Seleccionar imagen antigua');
125             if isequal(fileName, 0) || isequal(folderPath, 0)
126                 return; % El usuario canceló la selección
127             end
128             try
129                 fullImagePath = fullfile(folderPath, fileName);
130                 img = imread(fullImagePath);
131
132                 app.ImagenOriginal = img;
133                 app.ImagenProcesada = img;
134
135                 % Resetear historial
136                 app.HistorialImagenes = {img};
```

```
137                 app.HistorialNombres = {'Imagen original'};
138                 app.IndiceHistorial = 1;
139
140                 imshow(img, 'Parent', app.UIAxes);
141
142                 % Dibujar Histograma Original en UIAxes_3
143                 if size(img, 3) == 3
144                     grisOrig = rgb2gray(img);
145                 else
146                     grisOrig = img;
147                 end
148                 [countsOrig, binLocOrig] = imhist(grisOrig);
149                 cla(app.UIAxes_3);
150                 bar(app.UIAxes_3, binLocOrig, countsOrig, 'BarWidth', 1, 'FaceColor', [0 0.4470 0.7410],
                        'EdgeColor', 'none');
151                 app.UIAxes_3.XLim = [0 255];
152                 app.UIAxes_3.YLimMode = 'auto';
153                 app.UIAxes_3.YColor = [0.9 0.9 0.9];
154                 app.UIAxes_3.XColor = [0.9 0.9 0.9];
155
156                 % Actualizar visor del procesado con la misma imagen inicial
157                 app.actualizarInterfazProcesada();
158
159                 app.EstadodelaimagenLabel.Text = sprintf('Estado: Importación exitosa (%s)', fileName);
160             catch ME
161                 app.EstadodelaimagenLabel.Text = 'Estado: Fallo en la importación de la imagen.';
162             end
163         end
164
165         % Button pushed function: RayonesButton
166         function RayonesButtonPushed(app, event)
167             if isempty(app.ImagenProcesada)
168                 uialert(app.UIFigure, 'Por favor, importe una imagen primero.', 'Falta Imagen');
169                 return;
170             end
171
172             app.EstadodelaimagenLabel.Text = 'Estado: Eliminando rayones...';
173             drawnow;
174
175             % 1. Obtener imagen actual
176             img = app.ImagenProcesada;
177
178             % Convertir a escala de grises para la máscara
179             if size(img, 3) == 3
180                 grayImg = rgb2gray(img);
181             else
182                 grayImg = img;
183             end
184
185             % 2. Top-Hat morfológico con un elemento estructurante más amplio (para rayones gruesos)
186             se = strel('disk', 6); % Aumentado de 4 a 6
187             tophatImg = imtophat(grayImg, se);
188
189             % Umbral adaptativo basado en la mediana y desviación estándar del Top-Hat
190             umbral = median(tophatImg(:)) + 1.8 * std(double(tophatImg(:)));
191             mascaraRayones = tophatImg > umbral;
192
193             % Engrosar la máscara para cubrir bordes desgastados del rayón
194             mascaraRayones = imdilate(mascaraRayones, strel('disk', 2));
195
196             % 3. Aplicar Inpainting (regionfill) sobre la máscara generada
197             if size(img, 3) == 3
198                 imgRestaurada = img;
199                 for c = 1:3
200                     imgRestaurada(:,:,c) = regionfill(img(:,:,c), mascaraRayones);
201                 end
202             else
203                 imgRestaurada = regionfill(img, mascaraRayones);
204             end
205
206             % 4. Actualizar variable en proceso
207             app.ImagenProcesada = imgRestaurada;
208
209             % 5. Mostrar imagen procesada
```

```
            imshow(imgRestaurada, 'Parent', app.UIAxes_2);

            % 6. Renderizar Histograma Restaurado en UIAxes_4
            if size(imgRestaurada, 3) == 3
                grisHist = rgb2gray(imgRestaurada);
            else
                grisHist = imgRestaurada;
            end

            [counts, binLocations] = imhist(grisHist);
            bar(app.UIAxes_4, binLocations, counts, 'BarWidth', 1, 'FaceColor', [0.8 0.8 0.8], 'EdgeColor',
                'none');
            app.UIAxes_4.XLim = [0 255];
            app.UIAxes_4.YColor = [0.9 0.9 0.9];
            app.UIAxes_4.XColor = [0.9 0.9 0.9];

            % 7. Renderizar Histograma Original en UIAxes_3 si aún está vacío
            if size(app.ImagenOriginal, 3) == 3
                grisOrig = rgb2gray(app.ImagenOriginal);
            else
                grisOrig = app.ImagenOriginal;
            end
            [countsOrig, binLocOrig] = imhist(grisOrig);
            bar(app.UIAxes_3, binLocOrig, countsOrig, 'BarWidth', 1, 'FaceColor', [0.8 0.8 0.8], 'EdgeColor',
                'none');
            app.UIAxes_3.XLim = [0 255];
            app.UIAxes_3.YColor = [0.9 0.9 0.9];
            app.UIAxes_3.XColor = [0.9 0.9 0.9];

            app.guardarEstado(imgRestaurada);
            app.actualizarInterfazProcesada();
            app.EstadodelaimagenLabel.Text = 'Estado: Rayones eliminados correctamente.';
        end

        % Button pushed function: ArrugasPlieguesButton
        function ArrugasPlieguesButtonPushed(app, event)
            if isempty(app.ImagenProcesada)
                uialert(app.UIFigure, ...
                    'Por favor, importe una imagen primero.', ...
                    'Falta Imagen');
                return;
            end

            app.EstadodelaimagenLabel.Text = ...
                'Estado: Atenuando arrugas y pliegues (M2)...';
            drawnow;

            try
                % 2. Obtener la imagen actual
                img = app.ImagenProcesada;

                % 3. Definir el elemento estructurante
                se = strel('disk', 3);

                % 4. Aplicar cierre morfológico y filtro de mediana
                if size(img, 3) == 3

                    % Imagen a color: procesar cada canal RGB
                    imgRestaurada = img;

                    for c = 1:3

                        canal = img(:,:,c);

                        % Cierre morfológico
                        canal = imclose(canal, se);

                        % Filtro de mediana
                        canal = medfilt2(canal, [5 5], 'symmetric');

                        imgRestaurada(:,:,c) = canal;

                    end
```

```
                else

                    % Imagen en escala de grises
                    imgRestaurada = imclose(img, se);

                    imgRestaurada = medfilt2( ...
                        imgRestaurada, [5 5], 'symmetric');

                end

                % 5. Guardar el resultado en el historial
                app.guardarEstado(imgRestaurada, ...
                    'M2 - Arrugas y pliegues');

                % 6. Actualizar imagen e histogramas
                app.actualizarInterfazProcesada();

                % 7. Actualizar estado
                app.EstadodelaimagenLabel.Text = ...
                    'Estado: Arrugas y pliegues procesados correctamente.';

            catch ME

                uialert(app.UIFigure, ME.message, ...
                    'Error en M2');

                app.EstadodelaimagenLabel.Text = ...
                    'Estado: Error al procesar arrugas y pliegues.';

            end
        end

        % Button pushed function: DegradacinButton
        function DegradacinButtonPushed(app, event)

           if isempty(app.ImagenOriginal) && isempty(app.ImagenProcesada)
            uialert(app.UIFigure, 'Por favor, importe una imagen primero.', 'Falta Imagen');
            return;
           end

            app.EstadodelaimagenLabel.Text = 'Estado: Restaurando imagen degradada por el sol...';
            drawnow;

            try
                if ~isempty(app.ImagenProcesada)
                    imgBase = app.ImagenProcesada;
                else
                    imgBase = app.ImagenOriginal;
                end

                % 1. Restauración HSV / CLAHE
                if size(imgBase, 3) == 3
                    hsvImg = rgb2hsv(imgBase);
                    hsvImg(:,:,2) = min(hsvImg(:,:,2) * 1.3, 1.0);
                    hsvImg(:,:,3) = adapthisteq(hsvImg(:,:,3), 'ClipLimit', 0.02);
                    imgRestaurada = hsv2rgb(hsvImg);
                else
                    imgRestaurada = adapthisteq(imgBase, 'ClipLimit', 0.02);
                end

                % Filtro gaussiano suave
                imgRestaurada = imgaussfilt(imgRestaurada, 0.6);
                imgRestaurada = im2uint8(imgRestaurada);

                % 2. Actualizar propiedad en App Designer
                app.ImagenProcesada = imgRestaurada;

                % 3. Mostrar la imagen procesada
                imshow(imgRestaurada, 'Parent', app.UIAxes_2);

                % 4. Renderizar Histograma Restaurado en UIAxes_4
                if size(imgRestaurada, 3) == 3
                    grisHist = rgb2gray(imgRestaurada);
                else
```

```
                    grisHist = imgRestaurada;
                end

                [counts, binLocations] = imhist(grisHist);
                bar(app.UIAxes_4, binLocations, counts, 'BarWidth', 1, 'FaceColor', [0 0.4470 0.7410],
                    'EdgeColor', 'none');
                app.UIAxes_4.XLim = [0 255];
                app.UIAxes_4.YColor = [0.9 0.9 0.9];
                app.UIAxes_4.XColor = [0.9 0.9 0.9];

                % 5. Renderizar Histograma Original en UIAxes_3 si aplica
                if size(app.ImagenOriginal, 3) == 3
                    grisOrig = rgb2gray(app.ImagenOriginal);
                else
                    grisOrig = app.ImagenOriginal;
                end
                [countsOrig, binLocOrig] = imhist(grisOrig);
                bar(app.UIAxes_3, binLocOrig, countsOrig, 'BarWidth', 1, 'FaceColor', [0 0.4470 0.7410],
                    'EdgeColor', 'none');
                app.UIAxes_3.XLim = [0 255];
                app.UIAxes_3.YColor = [0.9 0.9 0.9];
                app.UIAxes_3.XColor = [0.9 0.9 0.9];

                app.guardarEstado(imgRestaurada);
                app.actualizarInterfazProcesada();
                app.EstadodelaimagenLabel.Text = 'Estado: Restauración por degradación solar completada.';

            catch ME
                app.EstadodelaimagenLabel.Text = 'Estado: Error al aplicar la restauración por degradación.';
            end
        end

        % Button pushed function: DeshacerButton
        function DeshacerButtonPushed(app, event)
            if app.IndiceHistorial > 1
                app.IndiceHistorial = app.IndiceHistorial - 1;
                app.ImagenProcesada = app.HistorialImagenes{app.IndiceHistorial};

                app.actualizarInterfazProcesada();

                app.EstadodelaimagenLabel.Text = sprintf('Estado: Operación deshecha (%d/%d).', ...
                    app.IndiceHistorial, length(app.HistorialImagenes));
            else
                uialert(app.UIFigure, 'No hay más operaciones para deshacer.', 'Historial Vacío');
            end
        end

        % Button pushed function: RehacerButton
        function RehacerButtonPushed(app, event)

                if app.IndiceHistorial < length(app.HistorialImagenes)
                    % Avanzar un estado en el historial
                    app.IndiceHistorial = app.IndiceHistorial + 1;
                    app.ImagenProcesada = app.HistorialImagenes{app.IndiceHistorial};

                    % Actualizar la imagen y el histograma UIAxes_4
                    app.actualizarInterfazProcesada();

                    app.EstadodelaimagenLabel.Text = sprintf('Estado: Operación rehecha (%d/%d).', ...
                        app.IndiceHistorial, length(app.HistorialImagenes));
                else
                    uialert(app.UIFigure, 'No hay más operaciones para rehacer.', 'Fin del Historial');
                end
        end

        % Button pushed function: ReconstruccinButton
        function ReconstruccinButtonPushed(app, event)
            if isempty(app.ImagenProcesada)
                uialert(app.UIFigure, 'Por favor, importe una imagen primero.', 'Falta Imagen');
                return;
            end

            app.EstadodelaimagenLabel.Text = 'Estado: Reconstruyendo roturas y esquinas (M4)...';
            drawnow;
```

```
428
429             try
430                 img = app.ImagenProcesada;
431
432                 if size(img, 3) == 3
433                     grayImg = rgb2gray(img);
434                 else
435                     grayImg = img;
436                 end
437
438                 % 1. Detección de fondo/papel faltante (zonas sobreexpuestas por escaneo > 225)
439                 mascaraVacios = (grayImg > 225);
440
441                 % 2. Limitar la reconstrucción de esquinas estrictamente al PERÍMETRO (25% exterior)
442                 % Esto aísla el centro de la imagen y protege ojos, nariz y boca
443                 [filas, columnas] = size(grayImg);
444                 mascaraPerimetro = false(filas, columnas);
445                 mF = round(filas * 0.25);
446                 mC = round(columnas * 0.25);
447                 mascaraPerimetro(1:mF, :) = true;
448                 mascaraPerimetro(end-mF+1:end, :) = true;
449                 mascaraPerimetro(:, 1:mC) = true;
450                 mascaraPerimetro(:, end-mC+1:end) = true;
451
452                 mascaraEsquinas = mascaraVacios & mascaraPerimetro;
453
454                 % 3. Detección de grietas morfológicas usando solo Top-Hat con alto umbral
455                 % Se remueve Bottom-Hat para evitar interpretar sombras oscuras del rostro como daños
456                 topHat = imtophat(grayImg, strel('disk', 4));
457                 umbralTopHat = median(topHat(:)) + 3.0 * std(double(topHat(:)));
458                 mascaraGrietas = topHat > umbralTopHat;
459
460                 % 4. Unir máscaras
461                 mascaraRoturas = mascaraEsquinas | mascaraGrietas;
462
463                 % 5. Limpieza y dilatación controlada
464                 mascaraRoturas = bwareaopen(mascaraRoturas, 30);
465                 mascaraRoturas = imdilate(mascaraRoturas, strel('disk', 3));
466
467                 if ~any(mascaraRoturas(:))
468                     app.EstadodelaimagenLabel.Text = 'Estado: No se detectaron roturas en la imagen.';
469                     return;
470                 end
471
472                 % 6. Inpainting canal por canal
473                 if size(img, 3) == 3
474                     imgRestaurada = img;
475                     for c = 1:3
476                         imgRestaurada(:,:,c) = regionfill(img(:,:,c), mascaraRoturas);
477                     end
478                 else
479                     imgRestaurada = regionfill(img, mascaraRoturas);
480                 end
481
482                 app.guardarEstado(imgRestaurada);
483                 app.actualizarInterfazProcesada();
484                 app.EstadodelaimagenLabel.Text = 'Estado: Reconstrucción M4 completada exitosamente.';
485
486             catch ME
487                 uialert(app.UIFigure, ME.message, 'Error en Reconstrucción M4');
488                 app.EstadodelaimagenLabel.Text = 'Estado: Error al ejecutar el módulo M4.';
489             end
490         end
491
492         % Button pushed function: AdicindeColorButton
493         function AdicindeColorButtonPushed(app, event)
494             if isempty(app.ImagenProcesada)
495                 uialert(app.UIFigure, 'Por favor, importe una imagen primero.', 'Falta Imagen');
496                 return;
497             end
498
499             app.EstadodelaimagenLabel.Text = 'Estado: Colorizando por transferencia de luminancia L*a*b*
                    (M6)...';
500             drawnow;
```

```
501
502             try
503                 img = app.ImagenProcesada;
504
505                 % 1. Extraer escala de grises / luminancia
506                 if size(img, 3) == 3
507                     grayImg = rgb2gray(img);
508                 else
509                     grayImg = img;
510                 end
511
512                 lum = im2double(grayImg);
513                 L = lum * 100; % Escala L* de 0 a 100
514
515                 % 2. Puntos de control para la curva de transferencia cromática (PDI determinista)
516                 % L_puntos: Niveles de brillo (0=Negro absoluto, 100=Blanco puro)
517                 % a_puntos: Verde (-) a Rojo (+)
518                 % b_puntos: Azul (-) a Amarillo (+)
519                 L_puntos = [0,  12,  30,  55,  75,  92, 100];
520                 a_puntos = [1,   2,   7,  13,   9,   2,   0];
521                 b_puntos = [2,   3,  10,  16,  11,   3,   1];
522
523                 % 3. Interpolación suave continua (pchip) para cada píxel de la matriz
524                 % Aplica el tono a TODAS las zonas según su brillo, sin máscaras circulares rígidas
525                 a = interp1(L_puntos, a_puntos, L, 'pchip');
526                 b = interp1(L_puntos, b_puntos, L, 'pchip');
527
528                 % 4. Filtrado suave morfológico para homogeneizar la transición cromática
529                 a = imgaussfilt(a, 1.5);
530                 b = imgaussfilt(b, 1.5);
531
532                 % 5. Reconstrucción de L*a*b* a RGB
533                 imgLAB = cat(3, L, a, b);
534                 imgRestaurada = lab2rgb(imgLAB);
535                 imgRestaurada = im2uint8(imgRestaurada);
536
537                 % 6. Actualizar historial e interfaz
538                 app.guardarEstado(imgRestaurada);
539                 app.actualizarInterfazProcesada();
540                 app.EstadodelaimagenLabel.Text = 'Estado: Colorización M6 completada exitosamente.';
541
542             catch ME
543                 uialert(app.UIFigure, ME.message, 'Error en Colorización M6');
544                 app.EstadodelaimagenLabel.Text = 'Estado: Error al ejecutar la colorización M6.';
545             end
546         end
547
548         % Button pushed function: ManchasButton
549         function ManchasButtonPushed(app, event)
550 if isempty(app.ImagenProcesada)
551         uialert(app.UIFigure, 'Por favor, importe una imagen primero.', 'Falta Imagen');
552         return;
553     end
554
555     app.EstadodelaimagenLabel.Text = 'Estado: Disminuyendo manchas (M3)...';
556     drawnow;
557
558     try
559         % --- Parámetros ajustables (calibrados para gotas en el lente) ---
560         kFuerte    = 3.0;    % sigmas para semillas (más bajo = más sensible)
561         kDebil     = 1.8;    % sigmas para extender la mancha (histéresis)
562         factorEsc  = 0.25;   % escala para estimar el fondo (rápido)
563         ventanaEsc = 25;     % impar; a escala 0.25 equivale a ~100 px reales
564         areaMin    = 30;     % px mínimos para considerar mancha
565         areaMaxFr  = 0.010;  % fracción máxima de la imagen por región
566         ratioMax   = 3;      % eje mayor/menor (mayor = línea/rayo, se protege)
567         solidezMin = 0.5;    % formas muy irregulares se protegen
568         radioDilat = 5;      % cubre el halo difuso de la gota
569         radioInp   = 15;     % radio de inpaintCoherent
570         mostrarMascara = false;  % true para depurar la máscara
571         % -----------------------------------------------------------------
572
573         % 0. Normalizar clase: elimina la dependencia de uint8 vs double
574         img = im2uint8(app.ImagenProcesada);
```

```
        esGris = size(img, 3) == 1;
        if esGris
            img = repmat(img, 1, 1, 3);
        end

        % 1. Lab: luminancia + croma
        lab = rgb2lab(img);
        L = lab(:,:,1);  A = lab(:,:,2);  B = lab(:,:,3);

        % 2. Fondo local robusto, estimado a escala reducida
        fondo = @(X) imresize( ...
                    medfilt2(imresize(X, factorEsc, 'bilinear'), ...
                             [ventanaEsc ventanaEsc], 'symmetric'), ...
                    [size(X,1), size(X,2)], 'bilinear');

        bgL = fondo(L);  bgA = fondo(A);  bgB = fondo(B);

        dL = L - bgL;
        dC = hypot(A - bgA, B - bgB);

        % 3. Puntaje normalizado (MAD); solo oscurecimientos en luminancia
        sL = max(1.4826 * median(abs(dL(:) - median(dL(:)))), 1);
        sC = max(1.4826 * median(dC(:)), 1);
        puntaje = max(max(-dL, 0) / sL, dC / sC);

        % 4. Histéresis: semillas fuertes extendidas a toda la mancha
        mascara = imreconstruct(puntaje > kFuerte, puntaje > kDebil);

        % 5. Filtrado por tamaño
        mascara = bwareaopen(mascara, areaMin);
        mascara = mascara & ~bwareaopen(mascara, round(areaMaxFr * numel(mascara)));

        % 6. Filtrado por forma: proteger líneas, rayo, arcoíris y estructuras alargadas
        cc = bwconncomp(mascara);
        if cc.NumObjects > 0
            st = regionprops(cc, 'Solidity', 'MajorAxisLength', 'MinorAxisLength');
            ratio = [st.MajorAxisLength] ./ max([st.MinorAxisLength], 1);
            conservar = ([st.Solidity] > solidezMin) & (ratio < ratioMax);
            mascara = ismember(labelmatrix(cc), find(conservar));
        end

        % 7. Expandir para cubrir halos de la mancha
        mascara = imdilate(mascara, strel('disk', radioDilat));

        nManchas = bwconncomp(mascara).NumObjects;
        if nManchas == 0
            app.EstadodelaimagenLabel.Text = ...
                'Estado: No se detectaron manchas (prueba bajar kFuerte).';
            return;
        end

        % Depuración visual opcional
        if mostrarMascara
            figure; imshow(labeloverlay(img, mascara, 'Transparency', 0.5));
        end

        % 8. Reparación
        try
            imgRestaurada = inpaintCoherent(img, mascara, ...
                'Radius', radioInp, 'SmoothingFactor', 2);   % Requiere R2019a+ o Computer Vision Toolbox
        catch
            imgRestaurada = img;
            for c = 1:3
                imgRestaurada(:,:,c) = regionfill(img(:,:,c), mascara);
            end
        end

        if esGris
            imgRestaurada = imgRestaurada(:,:,1);
        end

        % 9. Guardar y actualizar interfaz
        app.guardarEstado(imgRestaurada);
        app.actualizarInterfazProcesada();
```

```
        app.EstadodelaimagenLabel.Text = sprintf( ...
            'Estado: %d manchas reparadas (%.2f%% de la imagen).', ...
            nManchas, 100 * nnz(mascara) / numel(mascara));

    catch ME
        uialert(app.UIFigure, ME.message, 'Error en Módulo 3');
        app.EstadodelaimagenLabel.Text = 'Estado: Error al aplicar la disminución de manchas.';
    end
        end

        % Button pushed function: HistorialButton
        function HistorialButtonPushed(app, event)
            if app.IndiceHistorial == 0
                uialert(app.UIFigure, 'El historial está vacío. Importe y procese una imagen primero.',
                    'Historial Vacío');
                return;
            end

            % 1. Cerrar ventana previa si está abierta
            figExistente = findobj('Type', 'figure', 'Name', 'Historial de Operaciones');
            if ~isempty(figExistente)
                close(figExistente);
            end

            % 2. Crear ventana limpia (650x540)
            histFig = uifigure('Name', 'Historial de Operaciones', ...
                               'Position', [200, 150, 650, 540], ...
                               'Color', [0.949 0.9098 0.851]);

            numOperaciones = length(app.HistorialImagenes);
            itemsPorPagina = 3;
            totalPaginas = ceil(numOperaciones / itemsPorPagina);
            pagActual = 1;

            % 3. Grid principal (Fila 1: Miniaturas, Fila 2: Barra de Paginación)
            glMain = uigridlayout(histFig, [2, 1]);
            glMain.RowHeight = {'1x', 45};
            glMain.Padding = [10 10 10 10];

            % Container para las 3 imágenes
            panelPagina = uipanel(glMain, 'BackgroundColor', [0.949 0.9098 0.851], ...
                                          'BorderType', 'none');
            panelPagina.Layout.Row = 1;

            % Grid de 3 filas x 2 columnas para mostrar los 3 items grandes
            glItems = uigridlayout(panelPagina, [3, 2]);
            glItems.ColumnWidth = {200, '1x'};
            glItems.RowHeight = {140, 140, 140};
            glItems.Padding = [5 5 5 5];

            % 4. Barra de navegación inferior
            panelNav = uipanel(glMain, 'BackgroundColor', [0.949 0.9098 0.851], ...
                                       'BorderType', 'none');
            panelNav.Layout.Row = 2;

            glNav = uigridlayout(panelNav, [1, 3]);
            glNav.ColumnWidth = {110, '1x', 110};
            glNav.Padding = [0 5 0 5];

            btnPrev = uibutton(glNav, 'push', 'Text', '◄ Anterior', ...
                               'BackgroundColor', [0.0588 0.251 0.4], ...
                               'FontWeight', 'bold', 'FontColor', [0.949 0.9098 0.851]);
            btnPrev.Layout.Column = 1;

            lblPagina = uilabel(glNav, 'HorizontalAlignment', 'center', ...
                                'FontSize', 14, 'FontWeight', 'bold', ...
                                'FontColor', [0.0588 0.251 0.4]);
            lblPagina.Layout.Column = 2;

            btnNext = uibutton(glNav, 'push', 'Text', 'Siguiente ►', ...
                               'BackgroundColor', [0.0588 0.251 0.4], ...
                               'FontWeight', 'bold', 'FontColor', [0.949 0.9098 0.851]);
            btnNext.Layout.Column = 3;

```

```
            % Asignar callbacks a los botones de paginación
            btnPrev.ButtonPushedFcn = @(~,~) cambiarPagina(-1);
            btnNext.ButtonPushedFcn = @(~,~) cambiarPagina(1);

            % Renderizar primera página
            renderizarPagina();

            % --- Funciones internas de renderizado y control ---
            function renderizarPagina()
                % Limpiar grid de items
                delete(glItems.Children);

                inicioIdx = (pagActual - 1) * itemsPorPagina + 1;
                finIdx = min(inicioIdx + itemsPorPagina - 1, numOperaciones);

                filaGrid = 1;
                for i = inicioIdx:finIdx
                    % Miniatura grande
                    ax = uiaxes(glItems);
                    ax.Layout.Row = filaGrid;
                    ax.Layout.Column = 1;
                    ax.XTick = [];
                    ax.YTick = [];
                    ax.Toolbar.Visible = 'off';

                    imshow(app.HistorialImagenes{i}, 'Parent', ax); %#ok<IMSHOW>

                    % Texto descriptivo
                    lbl = uilabel(glItems);
                    lbl.Layout.Row = filaGrid;
                    lbl.Layout.Column = 2;
                    lbl.FontSize = 15;
                    lbl.FontWeight = 'bold';

                    if i <= length(app.HistorialNombres) && ~isempty(app.HistorialNombres{i})
                        nombreOp = app.HistorialNombres{i};
                    else
                        nombreOp = sprintf('Paso %d', i);
                    end

                    if i == app.IndiceHistorial
                        lbl.Text = sprintf('Paso %d: %s ◄ (Estado Actual)', i, nombreOp);
                        lbl.FontColor = [0.0588 0.251 0.4];
                    else
                        lbl.Text = sprintf('Paso %d: %s', i, nombreOp);
                        lbl.FontColor = [0.4 0.4 0.4];
                    end

                    filaGrid = filaGrid + 1;
                end

                % Actualizar controles del footer
                lblPagina.Text = sprintf('Página %d de %d', pagActual, totalPaginas);
                btnPrev.Enable = pagActual > 1;
                btnNext.Enable = pagActual < totalPaginas;
            end

            function cambiarPagina(delta)
                pagActual = pagActual + delta;
                renderizarPagina();
            end


        end

        % Button pushed function: ExportarButton
        function ExportarButtonPushed(app, event)
            if isempty(app.ImagenProcesada)
                uialert(app.UIFigure, ...
                    'Primero debe importar y procesar una imagen.', ...
                    'Sin imagen');
                return;
            end

```

```
            filtros = {
                '*.png', 'Imagen PNG (*.png)';
                '*.jpg', 'Imagen JPEG (*.jpg)';
                '*.tif', 'Imagen TIFF (*.tif)';
                '*.bmp', 'Imagen BMP (*.bmp)'
                };

            [fileName, folderPath, filtro] = uiputfile( ...
                filtros, 'Exportar imagen restaurada', ...
                'imagen_restaurada.png');

            if isequal(fileName, 0)
                return;
            end

            try

                [~, ~, extension] = fileparts(fileName);

                if isempty(extension)
                    extensiones = {'.png', '.jpg', '.tif', '.bmp'};
                    fileName = [fileName extensiones{filtro}];
                end


                img = app.ImagenProcesada;


                rutaCompleta = fullfile(folderPath, fileName);
                imwrite(img, rutaCompleta);


                app.EstadodelaimagenLabel.Text = ...
                    'Estado: Imagen exportada correctamente.';

                uialert(app.UIFigure, ...
                    sprintf('Imagen guardada correctamente en:\n%s', ...
                    rutaCompleta), ...
                    'Exportación exitosa');

            catch ME
                uialert(app.UIFigure, ME.message, ...
                    'Error al exportar');

                app.EstadodelaimagenLabel.Text = ...
                    'Estado: Error al exportar la imagen.';
            end
        end
    end

    % Component initialization
    methods (Access = private)

        % Create UIFigure and components
        function createComponents(app)

            % Get the file path for locating images
            pathToMLAPP = fileparts(mfilename('fullpath'));

            % Create UIFigure and hide until all components are created
            app.UIFigure = uifigure('Visible', 'off');
            app.UIFigure.Color = [0.0588 0.251 0.4];
            app.UIFigure.Position = [100 1 1090 851];
            app.UIFigure.Name = 'MATLAB App';

            % Create Image
            app.Image = uiimage(app.UIFigure);
            app.Image.Position = [37 708 110 127];
            app.Image.ImageSource = fullfile(pathToMLAPP, 'Recursos', 'logofondo.png');

            % Create Panel
            app.Panel = uipanel(app.UIFigure);
            app.Panel.BorderColor = [0.851 0.7804 0.6588];
            app.Panel.HighlightColor = [0.851 0.7804 0.6588];
```

```
            app.Panel.BackgroundColor = [0.949 0.9098 0.851];
            app.Panel.Position = [181 45 897 790];

            % Create UIAxes_2
            app.UIAxes_2 = uiaxes(app.Panel);
            app.UIAxes_2.XTick = [];
            app.UIAxes_2.YTick = [];
            app.UIAxes_2.Position = [480 335 387 321];

            % Create UIAxes
            app.UIAxes = uiaxes(app.Panel);
            app.UIAxes.XTick = [];
            app.UIAxes.YTick = [];
            app.UIAxes.Position = [25 335 387 321];

            % Create UIAxes_3
            app.UIAxes_3 = uiaxes(app.Panel);
            app.UIAxes_3.XTick = [];
            app.UIAxes_3.YTick = [];
            app.UIAxes_3.Position = [26 161 387 158];

            % Create UIAxes_4
            app.UIAxes_4 = uiaxes(app.Panel);
            app.UIAxes_4.XTick = [];
            app.UIAxes_4.YTick = [];
            app.UIAxes_4.Position = [480 160 387 158];

            % Create Label_2
            app.Label_2 = uilabel(app.Panel);
            app.Label_2.FontName = 'Bauhaus 93';
            app.Label_2.FontSize = 36;
            app.Label_2.FontWeight = 'bold';
            app.Label_2.FontColor = [0.2784 0.2196 0.1216];
            app.Label_2.Position = [24 729 98 47];
            app.Label_2.Text = 'Inicio';

            % Create Label
            app.Label = uilabel(app.Panel);
            app.Label.FontName = 'Bahnschrift';
            app.Label.FontSize = 14;
            app.Label.FontWeight = 'bold';
            app.Label.FontColor = [0.3882 0.3843 0.3765];
            app.Label.Position = [24 708 211 22];
            app.Label.Text = 'Restaura hoy, conserva siempre ';

            % Create OriginalLabel
            app.OriginalLabel = uilabel(app.Panel);
            app.OriginalLabel.FontName = 'Century Gothic';
            app.OriginalLabel.FontSize = 26;
            app.OriginalLabel.FontWeight = 'bold';
            app.OriginalLabel.FontColor = [0.2784 0.2196 0.1216];
            app.OriginalLabel.Position = [44 660 104 35];
            app.OriginalLabel.Text = 'Original';

            % Create ImagenRestauradaLabel
            app.ImagenRestauradaLabel = uilabel(app.Panel);
            app.ImagenRestauradaLabel.FontName = 'Century Gothic';
            app.ImagenRestauradaLabel.FontSize = 26;
            app.ImagenRestauradaLabel.FontWeight = 'bold';
            app.ImagenRestauradaLabel.FontColor = [0.2784 0.2196 0.1216];
            app.ImagenRestauradaLabel.Position = [502 660 256 35];
            app.ImagenRestauradaLabel.Text = 'Imagen Restaurada';

            % Create Label_3
            app.Label_3 = uilabel(app.Panel);
            app.Label_3.FontName = 'Century Gothic';
            app.Label_3.FontSize = 18;
            app.Label_3.FontWeight = 'bold';
            app.Label_3.FontColor = [0.2784 0.2196 0.1216];
            app.Label_3.Position = [44 312 105 24];
            app.Label_3.Text = 'Histograma';

            % Create Panel_2
            app.Panel_2 = uipanel(app.Panel);
```

```
            app.Panel_2.ForegroundColor = [0.2784 0.2196 0.1216];
            app.Panel_2.BackgroundColor = [0.949 0.9098 0.851];
            app.Panel_2.FontSize = 14;
            app.Panel_2.Position = [25 63 855 79];

            % Create HerramientasdeRestauracinLabel
            app.HerramientasdeRestauracinLabel = uilabel(app.Panel_2);
            app.HerramientasdeRestauracinLabel.BackgroundColor = [0.0588 0.251 0.4];
            app.HerramientasdeRestauracinLabel.FontSize = 14;
            app.HerramientasdeRestauracinLabel.FontWeight = 'bold';
            app.HerramientasdeRestauracinLabel.FontColor = [0.949 0.9098 0.851];
            app.HerramientasdeRestauracinLabel.Position = [0 56 854 22];
            app.HerramientasdeRestauracinLabel.Text = ' Herramientas de Restauración';

            % Create RayonesButton
            app.RayonesButton = uibutton(app.Panel_2, 'push');
            app.RayonesButton.ButtonPushedFcn = createCallbackFcn(app, @RayonesButtonPushed, true);
            app.RayonesButton.BackgroundColor = [0.0588 0.251 0.4];
            app.RayonesButton.FontWeight = 'bold';
            app.RayonesButton.FontColor = [0.949 0.9098 0.851];
            app.RayonesButton.Position = [19 16 100 23];
            app.RayonesButton.Text = 'Rayones';

            % Create ArrugasPlieguesButton
            app.ArrugasPlieguesButton = uibutton(app.Panel_2, 'push');
            app.ArrugasPlieguesButton.ButtonPushedFcn = createCallbackFcn(app, @ArrugasPlieguesButtonPushed,
                true);
            app.ArrugasPlieguesButton.BackgroundColor = [0.0588 0.251 0.4];
            app.ArrugasPlieguesButton.FontWeight = 'bold';
            app.ArrugasPlieguesButton.FontColor = [0.949 0.9098 0.851];
            app.ArrugasPlieguesButton.Position = [150 16 120 23];
            app.ArrugasPlieguesButton.Text = 'Arrugas / Pliegues';

            % Create ManchasButton
            app.ManchasButton = uibutton(app.Panel_2, 'push');
            app.ManchasButton.ButtonPushedFcn = createCallbackFcn(app, @ManchasButtonPushed, true);
            app.ManchasButton.BackgroundColor = [0.0588 0.251 0.4];
            app.ManchasButton.FontWeight = 'bold';
            app.ManchasButton.FontColor = [0.949 0.9098 0.851];
            app.ManchasButton.Position = [301 16 100 23];
            app.ManchasButton.Text = 'Manchas';

            % Create ReconstruccinButton
            app.ReconstruccinButton = uibutton(app.Panel_2, 'push');
            app.ReconstruccinButton.ButtonPushedFcn = createCallbackFcn(app, @ReconstruccinButtonPushed, true);
            app.ReconstruccinButton.BackgroundColor = [0.0588 0.251 0.4];
            app.ReconstruccinButton.FontWeight = 'bold';
            app.ReconstruccinButton.FontColor = [0.949 0.9098 0.851];
            app.ReconstruccinButton.Position = [429 16 106 23];
            app.ReconstruccinButton.Text = 'Reconstrucción';

            % Create DegradacinButton
            app.DegradacinButton = uibutton(app.Panel_2, 'push');
            app.DegradacinButton.ButtonPushedFcn = createCallbackFcn(app, @DegradacinButtonPushed, true);
            app.DegradacinButton.BackgroundColor = [0.0588 0.251 0.4];
            app.DegradacinButton.FontWeight = 'bold';
            app.DegradacinButton.FontColor = [0.949 0.9098 0.851];
            app.DegradacinButton.Position = [573 16 100 23];
            app.DegradacinButton.Text = 'Degradación';

            % Create AdicindeColorButton
            app.AdicindeColorButton = uibutton(app.Panel_2, 'push');
            app.AdicindeColorButton.ButtonPushedFcn = createCallbackFcn(app, @AdicindeColorButtonPushed, true);
            app.AdicindeColorButton.BackgroundColor = [0.0588 0.251 0.4];
            app.AdicindeColorButton.FontWeight = 'bold';
            app.AdicindeColorButton.FontColor = [0.949 0.9098 0.851];
            app.AdicindeColorButton.Position = [716 16 111 23];
            app.AdicindeColorButton.Text = 'Adición de Color';

            % Create ImportarButton
            app.ImportarButton = uibutton(app.Panel, 'push');
            app.ImportarButton.ButtonPushedFcn = createCallbackFcn(app, @ImportarButtonPushed, true);
            app.ImportarButton.BackgroundColor = [0.0588 0.251 0.4];
            app.ImportarButton.FontWeight = 'bold';
```

```
            app.ImportarButton.FontColor = [0.949 0.9098 0.851];
            app.ImportarButton.Tooltip = {'PNG, JPG/JPEG, TIFF y BMP'};
            app.ImportarButton.Position = [25 23 192 23];
            app.ImportarButton.Text = 'Importar';

            % Create DeshacerButton
            app.DeshacerButton = uibutton(app.Panel, 'push');
            app.DeshacerButton.ButtonPushedFcn = createCallbackFcn(app, @DeshacerButtonPushed, true);
            app.DeshacerButton.BackgroundColor = [0.0588 0.251 0.4];
            app.DeshacerButton.FontWeight = 'bold';
            app.DeshacerButton.FontColor = [0.949 0.9098 0.851];
            app.DeshacerButton.Tooltip = {'PNG, JPG/JPEG, TIFF y BMP'};
            app.DeshacerButton.Position = [302 23 110 23];
            app.DeshacerButton.Text = 'Deshacer';

            % Create RehacerButton
            app.RehacerButton = uibutton(app.Panel, 'push');
            app.RehacerButton.ButtonPushedFcn = createCallbackFcn(app, @RehacerButtonPushed, true);
            app.RehacerButton.BackgroundColor = [0.0588 0.251 0.4];
            app.RehacerButton.FontWeight = 'bold';
            app.RehacerButton.FontColor = [0.949 0.9098 0.851];
            app.RehacerButton.Tooltip = {'PNG, JPG/JPEG, TIFF y BMP'};
            app.RehacerButton.Position = [480 23 110 23];
            app.RehacerButton.Text = 'Rehacer';

            % Create ExportarButton
            app.ExportarButton = uibutton(app.Panel, 'push');
            app.ExportarButton.ButtonPushedFcn = createCallbackFcn(app, @ExportarButtonPushed, true);
            app.ExportarButton.BackgroundColor = [0.0588 0.251 0.4];
            app.ExportarButton.FontWeight = 'bold';
            app.ExportarButton.FontColor = [0.949 0.9098 0.851];
            app.ExportarButton.Position = [687 23 192 23];
            app.ExportarButton.Text = 'Exportar';

            % Create Image_2
            app.Image_2 = uiimage(app.UIFigure);
            app.Image_2.Position = [10 647 49 54];
            app.Image_2.ImageSource = fullfile(pathToMLAPP, 'Recursos', '2.png');

            % Create InicioButton
            app.InicioButton = uibutton(app.UIFigure, 'push');
            app.InicioButton.BackgroundColor = [0.949 0.9098 0.851];
            app.InicioButton.FontName = 'Century Gothic';
            app.InicioButton.FontSize = 18;
            app.InicioButton.FontWeight = 'bold';
            app.InicioButton.FontColor = [0.2784 0.2196 0.1216];
            app.InicioButton.Position = [58 647 112 54];
            app.InicioButton.Text = 'Inicio';

            % Create Image_3
            app.Image_3 = uiimage(app.UIFigure);
            app.Image_3.Position = [10 532 49 54];
            app.Image_3.ImageSource = fullfile(pathToMLAPP, 'Recursos', '4.png');

            % Create HistorialButton
            app.HistorialButton = uibutton(app.UIFigure, 'push');
            app.HistorialButton.ButtonPushedFcn = createCallbackFcn(app, @HistorialButtonPushed, true);
            app.HistorialButton.BackgroundColor = [0.949 0.9098 0.851];
            app.HistorialButton.FontName = 'Century Gothic';
            app.HistorialButton.FontSize = 18;
            app.HistorialButton.FontWeight = 'bold';
            app.HistorialButton.FontColor = [0.2784 0.2196 0.1216];
            app.HistorialButton.Position = [58 532 112 54];
            app.HistorialButton.Text = 'Historial';

            % Create Image_4
            app.Image_4 = uiimage(app.UIFigure);
            app.Image_4.Position = [10 413 49 54];
            app.Image_4.ImageSource = fullfile(pathToMLAPP, 'Recursos', '3.png');

            % Create AyudaButton
            app.AyudaButton = uibutton(app.UIFigure, 'push');
            app.AyudaButton.BackgroundColor = [0.949 0.9098 0.851];
            app.AyudaButton.FontName = 'Century Gothic';
```

```
            app.AyudaButton.FontSize = 18;
            app.AyudaButton.FontWeight = 'bold';
            app.AyudaButton.FontColor = [0.2784 0.2196 0.1216];
            app.AyudaButton.Position = [58 413 112 54];
            app.AyudaButton.Text = 'Ayuda';

            % Create EstadodelaimagenLabel
            app.EstadodelaimagenLabel = uilabel(app.UIFigure);
            app.EstadodelaimagenLabel.FontColor = [0.949 0.9098 0.851];
            app.EstadodelaimagenLabel.Position = [2 1 1089 45];
            app.EstadodelaimagenLabel.Text = ' Estado de la imagen';

            % Show the figure after all components are created
            app.UIFigure.Visible = 'on';
        end
    end

    % App creation and deletion
    methods (Access = public)

        % Construct app
        function app = app1

            % Create UIFigure and components
            createComponents(app)

            % Register the app with App Designer
            registerApp(app, app.UIFigure)

            if nargout == 0
                clear app
            end
        end

        % Code that executes before app deletion
        function delete(app)

            % Delete UIFigure when app is deleted
            delete(app.UIFigure)
        end
    end
end
```